# Questões sobre Programação Funcional

**Abstract**

Um conjunto de perguntas sobre a matéria de Programação Funcional. Esse Resumo foi feito com o link disponibilizado e respostas do Gemini.

**2. Em Haskell, o `$` é uma função que serve para determinar precedências, enquanto o `.` é uma função que acessa os atributos de um struct** Falso. O `$` é um operador de aplicação de função que serve para mudar a precedência, permitindo evitar parênteses. O `.` é um operador de composição de funções. Haskell não possui o conceito de `struct` como em outras linguagens.

**3. Em Java, os trechos funcionais são executados de forma lazy, já em Ruby, não.** Falso. Em Java, a avaliação lazy só ocorre em Streams. Em Ruby, a avaliação é estrita (eager) por padrão, mas pode-se usar bibliotecas para lazy evaluation.

**4. Dada a função `f :: Int -> Int`, definida como `f y = y * 2`. Se for aplicada `f 5` o resultado será 10.** Verdadeiro. A função multiplica o argumento por 2, então `5 * 2 = 10`.

**5. Haskell é uma linguagem multiparadigma, a qual permite o desenvolvimento de software de forma imperativa, orientada a objetos e funcional.** Falso. Haskell é uma linguagem **funcional pura**. Embora seja possível emular alguns comportamentos de outros paradigmas, ela não foi projetada para ser multiparadigma nesse sentido.

**6. A programação funcional não está diretamente relacionada ao cálculo lambda** Falso. A programação funcional está **diretamente relacionada** ao cálculo lambda. O cálculo lambda é a base teórica para a programação funcional.

**7. Funções anônimas são funções sem nome, geralmente usadas como parâmetros de funções.** Verdadeiro. Funções anônimas, também conhecidas como lambdas, são definidas sem um nome e frequentemente passadas como argumentos ou usadas em contextos onde uma função temporária é necessária.

**8. Em Haskell existem as instruções `if`, `then` e `else`.** Verdadeiro. Sim, Haskell possui as expressões `if`, `then` e `else` para controle de fluxo condicional.

**9. É aceitável declarar uma lista infinita em Haskell desde que o programa não precise utilizá-la por completo?** Verdadeiro. Sim, devido à **avaliação lazy** de Haskell, é perfeitamente aceitável e comum declarar listas infinitas, pois apenas os elementos necessários serão computados.

**10. O tipo Integer pode armazenar o fatorial de 13, enquanto o tipo Int não** Verdadeiro. O tipo `Integer` em Haskell pode armazenar números de precisão arbitrária, o que significa que ele pode lidar com o fatorial de 13 (que é 6.227.020.800). O tipo `Int` tem um limite de tamanho (geralmente equivalente ao tamanho de um inteiro nativo da máquina), e o fatorial de 13 provavelmente excederia esse limite em muitas arquiteturas.

**11. Ao criar um tipo de dados customizado, para usar o método `show` na visualização de informações das instâncias do novo tipo, é necessário também definir um mapeamento de representação da instância em string. A única forma possível de realizar tal mapeamento é utilizando deriving.** Falso. Embora `deriving` seja a forma mais comum e conveniente, não é a

única. É possível implementar a instância da **typeclass** `Show` manualmente para ter controle total sobre como o tipo é exibido.

**12. C é uma linguagem funcional.** Falso. C é uma linguagem **imperativa**, procedural.

**13. É possível passar uma função como argumento em C, utilizando ponteiros de função.** Verdadeiro. Sim, C permite o uso de **ponteiros de função**, o que possibilita passar funções como argumentos para outras funções.

**14. Em Java, mesmo em um trecho de código funcional, não será executado de forma preguiçosa (lazy).** Falso. Em Java, trechos de código funcional utilizando `Streams` podem ser executados de forma lazy.

**15. Uma lista declarada como `[3,5..]` começa no 3 e vai de 5 em 5 até o infinito** Falso. A lista `[3,5..]` começa no 3 e o próximo elemento é 5, indicando um incremento de `5 - 3 = 2`. Portanto, a lista gerada seria `[3, 5, 7, 9, ...]`.

**16. A linguagem Haskell possui avaliação lazy, ou seja, as expressões são avaliadas somente quando seu valor é realmente necessário.** Verdadeiro. Essa é uma definição correta de **avaliação lazy** em Haskell.

**17. Haskell possui suporte nativo para funções lambda?** Verdadeiro. Sim, **funções lambda** são uma característica fundamental de Haskell, expressas com a sintaxe `\` (barra invertida).

**18. Tupla é um tipo de estrutura de dados do Python que tem como principal característica ser imutável** Verdadeiro. Sim, **tuplas** em Python são estruturas de dados que, uma vez criadas, não podem ser modificadas (são imutáveis).

**19. A função `f = [n * (n + 1)div2 | n <- [1..]]` retorna a lista `[1, 3, 6, 10, 15 ...]`.** Verdadeiro. Esta é a fórmula para os **números triangulares**. Para `n = 1`, `1 * (1 + 1) / 2 = 1` Para `n = 2`, `2 * (2 + 1) / 2 = 3` Para `n = 3`, `3 * (3 + 1) / 2 = 6` E assim por diante.

**20. Ao invés de usar recursão, como é comum em paradigmas imperativos, a programação funcional é baseada na utilização de laços para a construção de programas** Falso. Na programação funcional, especialmente em linguagens puras como Haskell, a **recursão** é a técnica primária para iteração, e não laços imperativos.

**21. Na programação funcional não é possível realizar atividades repititivas, porque não se usa a estrutura `for`** Falso. É possível realizar atividades repetitivas usando **recursão**, funções de ordem superior (como `map`, `filter`, `fold`), e outros recursos específicos do paradigma funcional. O que não se usa são laços imperativos como o `for`.

**22. `filtro :: (a -> Bool) -> [a] -> [a]`. Essa função retorna uma lista que contém apenas os elementos da lista de entrada para os quais a função de teste retorna true** Verdadeiro. Esta é a assinatura de tipo para a função `filter` em Haskell, e sua descrição está correta. Ela recebe um predicado (função `a -> Bool`) e uma lista `[a]`, retornando uma nova lista com os elementos que satisfazem o predicado.

**23. Ruby off rails foi o framework que fez Ruby se tornar popular?** Falso. Foi o **Ruby on Rails** (e não `Ruby off rails`) que contribuiu significativamente para a popularidade da linguagem Ruby.

**24. A função `.` (ponto), utilizada para composição de funções, tem tipo `(a->b) -> (a->b) -> (a->b)`** Falso. O tipo correto do operador de **composição de funções** `.` em Haskell é `(b -> c) -> (a -> b) -> (a -> c)`. Ele compõe duas funções, `f` e `g`, de forma que `(f . g) x` é equivalente a `f (g x)`.

**25. Ruby on Rails não deu certo por conta da demora e dificuldade para construir projetos.** Falso. Ruby on Rails teve e continua tendo um sucesso considerável, sendo conhecido justamente pela sua rapidez e facilidade em construir projetos web, especialmente no início de seu ciclo de vida.

**26. O `where` serve para localizar uma função dentro de outra.** Falso. O `where` é usado para definir **funções locais** ou variáveis dentro de uma função principal, não para `localizar` uma função existente dentro de outra.

**27. Todo e qualquer problema é melhor resolvido por programação funcional.** Falso. Não. Embora a programação funcional seja poderosa para muitos tipos de problemas, nenhum paradigma de programação é a solução ideal para **todos** os problemas. A escolha do paradigma depende da natureza do problema, dos requisitos de desempenho, da familiaridade da equipe, etc.

**28. Programação funcional utiliza de recursão?** Verdadeiro. Sim, **recursão** é uma técnica fundamental e amplamente utilizada na programação funcional.

**29. A diferença entre Let e Where é que: o let vincula expressões a si e o where vincula somente construtores sintáticos.** Falso. Tanto `let` quanto `where` são usados para vinculação (binding) de nomes a expressões (valores ou funções). A principal diferença é o escopo e a ordem em que são escritos: `where` define vinculações locais no final de uma função, enquanto `let` pode ser usado em qualquer expressão para definir vinculações locais.

**30. Em haskell, toda expressão e função tem que retornar alguma coisa.** Verdadeiro. Sim, em Haskell, tudo é uma expressão e, portanto, toda expressão e função avalia para um valor (retorna alguma coisa). Mesmo efeitos colaterais são encapsulados em tipos como `IO`.

**31. O cálculo do comprimento de uma lista genérica, pode ser definido por uma função `len`, a partir das cláusulas `len [ ] = 0` e `len (h:t) = 1 + len(h)`.** Falso. A definição `len (h:t) = 1 + len(h)` está incorreta. Ela tenta somar 1 ao comprimento da cabeça (`h`), o que não faz sentido. O correto para calcular o comprimento da lista é `len (h:t) = 1 + len(t)`, ou seja, 1 mais o comprimento da cauda (`t`).

**32. A função dólar, definida como `f ($) x = f x` é utilizada como alternativa aos parênteses na definição da precedência de aplicações de funções.** Falso. A definição `f ($) x = f x` está sintaticamente incorreta e não representa o operador `$` em Haskell. O operador `$` tem tipo `(a -> b) -> a -> b` e é usado para aplicar uma função a um argumento com a menor precedência, eliminando a necessidade de parênteses em muitos casos. Por exemplo, `f $ x + y` é o mesmo que `f (x + y)`.

**33. Em geral, o operador `$` é usado para composição de funções, enquanto o operador `.` é usado para aplicação de funções, permitindo combinar e aplicar funções de diferentes maneiras.** Falso. É o oposto. O operador `$` é usado para **aplicação** de funções (com precedência mais baixa), e o operador `.` é usado para **composição** de funções.

**34. A programação funcional incentiva o uso de expressões lambda, que são funções anônimas que podem ser passadas como argumentos ou retornadas por outras funções.** Verdadeiro. Sim, o uso de **expressões lambda** (funções anônimas) é uma prática comum e incentivada na programação funcional.

**35. `deriving(Read)`, após a definição de um tipo de dados, permite gerar estruturas daquele dado a partir de strings. Para isso, a representação do dado em string deve ser idêntica à representação exibida pela função `show`.** Verdadeiro. Sim, o `deriving(Read)` gera uma instância da **typeclass `Read`** que permite converter strings em valores do tipo. Para que isso funcione corretamente, a string de entrada deve estar no formato que seria produzido pela função `show` para aquele tipo.

**36. Se o método `show` em Haskell faz a amostragem de um tipo, ele pode fazer a amostragem de uma lista do tipo, um exemplo seriam os inteiros** Verdadeiro. Sim, se um tipo tem uma instância de `Show`, uma lista desse tipo também pode ser `mostrada` (convertida para string) usando a função `show`. Por exemplo, `show [1, 2, 3]` funciona.

**37. A `Lazy Computation` consiste em apenas avaliar trechos do código que serão absolutamente necessários a fim de executar o algoritmo.** Verdadeiro. Essa é uma boa definição de **avaliação lazy**.

**38. Programação funcional, programação orientada a objetos e programação lógica são exemplos de paradigmas de programação declarativos.** Falso. A programação funcional e a programação lógica são paradigmas **declarativos**. A programação orientada a objetos (POO) é um paradigma **imperativo**.

**39. Em Haskell, é possível definir funções de alta ordem, ou seja, que recebem outras funções como argumentos ou retornam funções como resultado.** Verdadeiro. Sim, **funções de alta ordem** são uma característica central de Haskell.

**40. Java só é lazy quando se manuseiam streams** Verdadeiro. De forma geral, sim. A avaliação lazy em Java é predominantemente associada ao uso de Streams e operações intermediárias nelas.

**41. Dada a função `s x y = x + y` com `x` e `y` inteiros, sua definicão de tipo é `s :: Int -> (Int -> Int)`** Verdadeiro. Sim, essa é a forma **curried** do tipo, o que é padrão em Haskell. A função `s` recebe um `Int` e retorna uma função que espera outro `Int` e, finalmente, retorna um `Int`.

**42. A função `f s = s == reverse s` retorna verdadeiro para a String `arara` e falso para a string `papagaio`** Verdadeiro. A função `reverse` inverte a string. `arara` invertido é `arara`, então `s == reverse s` é verdadeiro. `papagaio` invertido é `oiagapap`, que não é igual a `papagaio`, então é falso.

**43. Uma função da typeclass `Show` pode ser printada** Falso. Não diretamente. A typeclass `Show` define o método `show` que converte um valor para uma `String`. Para `printar` algo no terminal, você geralmente precisa de uma função que lide com `IO`, como `print` ou `putStrLn`, que internamente usam `show` para formatar o valor.

**44. Em um programa feito na linguagem Haskell, nem todas as cláusulas precisam ser comparáveis** Falso. Todas as cláusulas de uma função devem ter um tipo consistente. Se `comparáveis` se refere a ter uma instância da typeclass `Eq`, então sim, nem todos os tipos precisam ter `Eq`. No entanto, todas as cláusulas de uma função devem ser válidas e coerentes com a definição geral da função.

**45. A filtragem de objetos com parâmetros específicos em Java é feita através do método `filter()`** Verdadeiro. Sim, em Java 8+ com Streams, o método `filter()` é usado para filtrar elementos com base em um predicado.

**46. As funções locais definidas com `where` não são acessíveis fora da função principal onde foram declaradas.** Verdadeiro. Sim, funções definidas em um bloco `where` têm escopo local à função onde foram declaradas.

**47. A programação funcional não permite o efeito colateral, o que significa que uma função pura sempre produzirá o mesmo resultado para a mesma entrada.** Falso. A programação funcional **incentiva** a ausência de efeitos colaterais e a utilização de **funções puras**. No entanto, linguagens funcionais podem ter mecanismos para lidar com efeitos colaterais (como `IO` em Haskell), e nem todas as linguagens funcionais são puras.

**48. Na programação funcional, é comum usar recursão em vez de laços de repetição para realizar iterações.** Verdadeiro. Sim, **recursão** é a abordagem preferida para iteração em programação funcional.

**49. A programação funcional é um paradigma de programação que enfatiza o uso de funções puras e evita o uso de estados mutáveis.** Verdadeiro. Essa é uma definição fundamental da **programação funcional**.

**50. Em Haskell, `F x = G 5 x` é o mesmo que `F = G 5`.** Verdadeiro. Sim, devido à **currying** e **aplicação parcial de funções** em Haskell, essas duas declarações são equivalentes. Se `G` tem o tipo `Int -> a -> b`, então `G 5` retorna uma função do tipo `a -> b`, que é exatamente o que `F` se torna.

**51. De forma simplificada, Lazy Evaluation consiste em esperar para processar uma expressão (ou parte dela) até o exato momento em que o resultado é utilizado.** Verdadeiro. Essa é uma definição concisa e precisa de **avaliação lazy**.

**52. Eq em Haskell é uma class type ou tipo de tipo** Verdadeiro. `Eq` é uma **typeclass**, que é um conceito que agrupa tipos que se comportam de uma certa maneira (neste caso, que podem ser comparados quanto à igualdade). Em Haskell, typeclasses são frequentemente descritas como `interfaces` ou `tipos de tipos`, no sentido de que definem um conjunto de operações que os tipos devem suportar.

**53. `len [] = 0 len(h:t) = 1+ len t` O codigo acima calcula o tamanho de uma lista?** Verdadeiro. Sim, essa é a definição padrão e correta de uma função recursiva para calcular o comprimento de uma lista em Haskell.

**54. A definição da cláusula `fat n = n * fat n - 1` em haskell é suficiente para calcular o fatorial de qualquer número n.** Falso. Esta definição está incorreta e causaria um loop infinito (ou um erro de runtime devido a argumentos negativos) porque falta a **condição de parada** (caso base) para a recursão (e o `- 1` está fora da chamada recursiva). A definição correta seria algo como: `fat 0 = 1` e `fat n = n * fat (n - 1)`.

**55. Funções de alta ordem são funções que podem receber outras funções como parâmetro** Verdadeiro. Sim, essa é uma das características das **funções de alta ordem** (também podem retornar funções como resultado).

**56. Laziness é um conceito intrínseco à toda linguagem funcional** Falso. Não. Embora seja uma característica marcante de linguagens como Haskell, muitas linguagens funcionais (como Scheme, OCaml, F#) utilizam avaliação estrita (eager evaluation) por padrão.

**57. A stream em Java segue a `lazy evaluation`, ou seja, passa os objetos um a um apenas quando `solicitado`** Verdadeiro. Sim, as operações intermediárias em Streams de Java são lazy; elas são executadas apenas quando uma operação terminal é chamada e os elementos são realmente necessários.

**58. o Maybe é uma abordagem que evita o uso de if-else em programação funcional** Falso. O tipo `Maybe` é uma forma de lidar com a ausência de um valor (similar a `null` em outras linguagens, mas de forma segura e tipada), evitando exceções de ponteiro nulo. Embora possa reduzir a necessidade de alguns `if-else`s para checagem de nulos, ele não `evita` o uso de `if-else` de forma geral. Ele é frequentemente usado com **pattern matching** ou funções como `maybe` que se comportam de forma similar a `if-else` para lidar com os casos `Just` e `Nothing`.

**59. A linguagem de haskell é fortemente tipada?** Verdadeiro. Sim, Haskell é uma linguagem **fortemente tipada** e usa inferência de tipos.

**60. Em Haskell, é possível usar funções dentro de comparação para retornar um valor booleano.** Verdadeiro. Sim, é comum e esperado usar funções que retornam `Bool` em comparações, como `(==)`, `(>)`, `elem`, etc.

**61. As linguagens funcionais possibilitam escrever códigos curtos, rápidos e de fácil manutenção.** Verdadeiro. Sim, a programação funcional pode levar a códigos mais concisos, menos

propensos a bugs (devido à imutabilidade e funções puras) e, consequentemente, mais fáceis de manter. A rapidez depende da implementação e otimização do compilador/interpretador.

**62. Linguagens funcionais não permitem a implementação de métodos de ordenação** Falso. Não, linguagens funcionais são perfeitamente capazes de implementar **métodos de ordenação**; na verdade, muitos algoritmos de ordenação (como quicksort e mergesort) têm implementações recursivas elegantes em linguagens funcionais.

**63. A implementação mais próxima do paradigma funcional em C está relacionada ao uso de ponteiro para funções.** Verdadeiro. Sim, **ponteiros para funções** em C permitem passar funções como argumentos, um conceito fundamental na programação funcional.

**64. Haskell tem uma sintaxe bidimensional** Falso. Haskell tem uma **sintaxe linear**, baseada em texto. O termo `sintaxe bidimensional` é mais associado a linguagens visuais ou notações matemáticas que usam arranjos espaciais.

**65. Como em Haskell não é possível alterar o valor de variáveis, não existem structs** Falso. Haskell não tem `variáveis` mutáveis no sentido imperativo, mas possui **tipos de dados** que são análogos a `structs` (registros), onde você agrupa diferentes tipos de dados. Esses tipos são imutáveis.

**66. O código Haskell: `x=[1,5..100]`, gera uma lista que vai de 1 até 100, com um intervalo de 4 em 4?** Verdadeiro. Sim. `[1, 5..100]` indica que a lista começa em 1, o próximo elemento é 5 (diferença de 4), e continua aumentando de 4 em 4 até 100. Então seria `[1, 5, 9, ..., 97]`.

**67. Um dos problemas em Haskell é que a linguagem não permite a definição de tipos de dados, os quais podem ter múltiplos construtores e parâmetros.** Falso. Não, isso não é um problema. Haskell permite a definição de **tipos de dados algébricos (ADTs)** que podem ter múltiplos construtores e cada construtor pode ter parâmetros. Isso é uma característica poderosa da linguagem.

**68. A recursão é uma das técnicas em programação funcional para suprir a falta de laços iterativos.** Verdadeiro. Sim, a **recursão** é a principal técnica de iteração na programação funcional.

**69. Dada a função `f x y = x + y`, o seu tipo é `f :: Num a => a -> a -> a`, podendo ser utilizada tanto com números inteiros quanto em reais.** Verdadeiro. Sim, a restrição de tipo `Num a` indica que `a` pode ser qualquer tipo numérico (como `Int`, `Integer`, `Float`, `Double`), e a função `+` opera sobre eles.

**70. A função: `p = f [2..] where f (p:xs) = p : f [x | x <- xs, xmodp /= 0]` retorna uma lista que pode ser impressa sem correr o risco de entrar em loop infinito, devido à Haskell ser lazy** Verdadeiro. Sim, essa é uma implementação do **crivo de Eratóstenes** para gerar números primos. Devido à **avaliação lazy** de Haskell, a lista de primos será gerada sob demanda, o que significa que você pode imprimir uma parte finita dela sem que o programa tente gerar todos os primos até o infinito.

**71. Na chamada `lazy evaluation`, todas as expressões são calculadas imediatamente, mesmo que seus resultados não sejam utilizados** Falso. É o oposto. Na avaliação lazy, as expressões são calculadas **somente** quando seus resultados são realmente necessários.

**72. Em Haskell, tanto o `$` quanto o `.` são funções que recebem uma função como argumento e retornam uma função** Falso. O `.` (composição) recebe duas funções e retorna uma função. O `$` (aplicação) recebe uma função e um valor, retornando um valor.

**73. A expressão `where` em haskell é usada para definir funções locais dentro de outras funções, permitindo a criação de funções auxiliares dentro de uma função principal** Verdadeiro. Sim, essa é a principal finalidade do **`where`**.

**74. Em C, na expressão `int (*filtro) (struct Aluno)`, o parâmetro `filtro` é um ponteiro para uma função que recebe uma struct Aluno como parâmetro e retorna um inteiro.** Verdadeiro. Sim, essa é a sintaxe correta para declarar um **ponteiro para uma função** em C.

**75. A `main` em Haskell é do tipo IO, ou seja, pode `mudar o mundo`** Verdadeiro. Sim, a função `main` em Haskell deve ter o tipo `IO ()` ou um tipo `IO a` para indicar que ela realiza ações de I/O (efeitos colaterais que `mudam o mundo` fora do programa).

**76. O `case` em haskell é amplamente utilizado para realizar expressões de casamento de padrões em listas e tuplas** Verdadeiro. Sim, o `case` é uma ferramenta poderosa para **casamento de padrões (pattern matching)** em Haskell e é muito usado com listas, tuplas, e outros tipos de dados algébricos.

**77. A função `f s = p == reverse p where p = map toLower $ filter isAlphaNum s`, quando aplicada à string `A man, a plan, a canal, Panama!`, retorna:** Verdadeiro. A função remove pontuação e espaços, converte para minúsculas e então checa se é um palíndromo. `A man, a plan, a canal, Panama!` se torna `amanaplanacanalpanama`, que é um palíndromo. Portanto, a função retornaria `True`.

**78. Ruby é uma linguagem de programação preguiçosa.** Falso. Ruby é uma linguagem de **avaliação estrita (eager evaluation)** por padrão.

**79. Linguagens de programação que usam o paradigma funcional devem implementar o Lazy Evaluation.** Falso. Não, a **avaliação lazy** é uma característica de algumas linguagens funcionais (como Haskell), mas não é um requisito para todas. Muitas linguagens funcionais usam avaliação estrita.

**80. Haskell, java com strings e JavaScript são todas linguagens `preguiçosas`.** Falso. Haskell é `preguiçosa` (lazy). Java (especialmente com Strings) e JavaScript são linguagens de **avaliação estrita (eager evaluation)** por padrão.

**81. Maybe é uma maneira de lidar com o valor nulo ou vazio em programação funcional** Verdadeiro. Sim, o tipo `Maybe` em Haskell é uma forma tipada e segura de representar a presença ou ausência de um valor, evitando o problema de `null` de outras linguagens.

**82. Haskell é a única linguagem que utiliza funções puras.** Falso. Não, muitas linguagens (e não apenas funcionais) têm suporte para **funções puras** ou encorajam seu uso, e existem outras linguagens funcionais puras além de Haskell.

**83. O principal aspecto da programação funcional é ser baseada em funções puras.** Verdadeiro. Sim, a ênfase em **funções puras** (funções sem efeitos colaterais e que produzem o mesmo resultado para as mesmas entradas) é um dos pilares da programação funcional.

**84. Haskell suporta herança de classes?** Falso. Haskell não suporta **herança de classes** no sentido da programação orientada a objetos (hierarquia de classes). Ele usa **typeclasses** para polimorfismo ad-hoc, que são diferentes da herança de classes.

**85. Linguagens funcionais são especialmente adequadas para lidar com a concorrência de recursos em sistemas distribuídos.** Verdadeiro. Sim, a **imutabilidade** e a ausência de efeitos colaterais em linguagens funcionais tornam-nas naturalmente adequadas para programação concorrente e paralela, pois eliminam muitos problemas de estado compartilhado e corrida de dados.

**86. na linguagem haskell, o tipo Int é geralmente mais eficiente em termos de uso de memória e desempenho em comparação com Integer** Verdadeiro. Sim, `Int` mapeia para o tipo inteiro nativo da máquina, o que geralmente o torna mais eficiente em termos de memória e desempenho do que `Integer`, que pode representar números de precisão arbitrária e, portanto, requer mais overhead.

**87. Closure é um conceito usado apenas em linguagens de programação funcional** Falso. Não, **closures** são um conceito presente em muitas linguagens de programação, incluindo linguagens imperativas e orientadas a objetos que suportam funções de primeira classe (como JavaScript, Python, Java 8+, C#).

**88. Haskell é uma linguagem em que tudo é função.** Falso. Haskell é uma linguagem onde tudo é uma **expressão** que avalia para um **valor**, e funções são valores de primeira classe. No entanto, existem também **tipos de dados**, não apenas funções.

**89. A programação funcional é um paradigma de programação que se baseia principalmente em estruturas de repetição, como laços for e while.** Falso. É o oposto. A programação funcional minimiza ou evita o uso de estruturas de repetição imperativas (`for`, `while`) e favorece a **recursão** e **funções de ordem superior**.

**90. Diferentemente da programação funcional em haskell, em java existe uma função `peek` que pode ser usada para depuração ou para observar o fluxo de elementos sem modificá-los.** Verdadeiro. Sim, em Java Streams, o método `peek()` é uma operação intermediária que permite realizar uma ação em cada elemento sem consumir a stream, sendo útil para depuração. Haskell tem suas próprias formas de depuração, mas o conceito de `peek` em um pipeline lazy é específico de como Streams funcionam em Java.

**91. Suponha uma função na linguagem C, `function(int var);`. Em seguida, na main foi escrito a seguinte linha, `int x = 5; if(function(x) == function(x)){...}`. Nesse caso, o código sempre executará o código dentro desse if.** Falso. Não necessariamente. Se `function` tiver **efeitos colaterais** (como modificar uma variável global, ler de um arquivo, gerar um número aleatório), `function(x)` pode retornar um valor diferente a cada chamada, fazendo com que a condição do `if` seja falsa. Se a função for pura, então sim, sempre será verdadeiro. A pergunta não especifica que a função é pura.

**92. É possível implementar conceitos funcionais em versões de linguagens imperativas modernas, como Javascript, Java e C++** Verdadeiro. Sim, muitas linguagens imperativas modernas incorporaram **recursos funcionais** (funções lambda, funções de ordem superior, Streams, etc.), permitindo que os desenvolvedores adotem um estilo de programação mais funcional.

**93. A programação funcional é exclusiva de linguagens de programação funcionais, como Haskell e Lisp, e não pode ser usada em linguagens imperativas.** Falso. Não, embora a programação funcional tenha suas raízes em linguagens funcionais, muitos de seus conceitos podem ser aplicados e implementados em linguagens imperativas modernas, como mencionado na questão 92.

**94. Select de ruby tem função similar a filter de Haskell.** Verdadeiro. Sim, o método `select` em Ruby (para arrays e enumeráveis) funciona de forma muito similar ao `filter` em Haskell, criando uma nova coleção contendo apenas os elementos para os quais um bloco de código (predicado) retorna verdadeiro.

**95. Em Haskell, se uma função imprime algo no terminal, essa função deve retornar um tipo IO** Verdadeiro. Sim, qualquer função em Haskell que realiza operações de I/O (como imprimir no terminal) deve ter um tipo de retorno encapsulado no **monad `IO`** para indicar que ela tem efeitos colaterais.

**96. `function' :: (Num b) => [a] -> b \n function' [] = 0 \n function' (_:xs) = 1 + function' xs \n` Essa função calcula o comprimento de uma lista.** Verdadeiro. Sim, essa função calcula o comprimento de uma lista. A restrição `Num b` e o retorno `b` permitem que o comprimento seja representado por qualquer tipo numérico.

**97. A função lambda sempre requer a especificação do tipo de seus argumentos** Falso. Não, em linguagens com **inferência de tipos** (como Haskell), a maioria das funções lambda não requer

que você especifique explicitamente os tipos de seus argumentos; o compilador pode inferi-los. Em outras linguagens (como Java), pode ser necessário.

**98. Java e C++ possuem possibilidade funcional, apesar de C++ ser uma opção melhor para o uso funcional** Falso. Ambas possuem possibilidades funcionais com recursos modernos. Qual é `melhor` é subjetivo e depende do contexto e preferências, mas C++ oferece mais controle de baixo nível, enquanto Java oferece uma abordagem mais de alto nível com Streams. Não há uma clara superioridade geral para `uso funcional`.

**99. A ordem de consideração dos parâmetros de uma função em haskell é sempre da esquerda para a direita?** Verdadeiro. Sim, em Haskell, a aplicação de funções e a **currying** ocorrem da esquerda para a direita.

**100. É possível chamar outra função dentro de uma função diferente?** Verdadeiro. Sim, é fundamental para a modularidade e composição de programas em qualquer linguagem de programação.

**101.** `const f = (x) =>{ return x + 42; } const f = (x) => (x + 42); const f = x => x + 42;` **As declarações acima são equivalentes?** Verdadeiro. Sim, em JavaScript, todas essas declarações de **arrow functions** são equivalentes e definem a mesma função. A sintaxe mais curta `x => x + 42` é uma forma abreviada para funções com um único parâmetro e uma única expressão de retorno.

**102. A programação funcional se baseia no uso de funções como parte fundamental na construção de programas** Verdadeiro. Sim, **funções** são os blocos de construção primários na programação funcional.

**103. A programação funcional em Java também é lazyness.** Falso. A programação funcional em Java (usando Streams) pode ser lazy para operações intermediárias, mas o Java em si não é intrinsecamente lazy em todo o seu escopo, como Haskell.

**104. Haskell é uma linguagem de programação compilada?** Verdadeiro. Sim, Haskell é uma linguagem **compilada** (geralmente para código nativo) e também pode ser executada por um interpretador.

**105. A função lambda é definida diretamente na expressão em que será usada, não precisando de palavras-chave como `def` ou `function`** Falso. Embora funções lambda sejam frequentemente definidas `inline`, a necessidade de palavras-chave depende da linguagem. Em Haskell, usa-se \ (barra invertida); em Python, `lambda`; em JavaScript, `function` ou a sintaxe de arrow function `=>`.

**106. Programação funcional consiste em passar trechos de código que se repetem como parâmetro de uma função,fazendo com que o código se torne mais flexível e otimizado.** Verdadeiro. Isso descreve bem o conceito de **funções de ordem superior**, que é um aspecto importante da programação funcional e leva a código mais flexível e reutilizável. A otimização pode vir disso, mas não é a única razão.

**107. Soluções recursivas são padrão em linguagens funcionais** Verdadeiro. Sim, a **recursão** é a forma idiomática de iteração e repetição na maioria das linguagens funcionais.

**108. A avaliação preguiçosa (lazy) em Haskell permite que todas as expressões sejam avaliadas imediatamente, mesmo que não sejam necessárias para o resultado final do programa?** Falso. É o oposto. A **avaliação preguiçosa** significa que as expressões são avaliadas **somente** quando seus valores são necessários para o resultado final do programa.